



საქართველოს ტექნიკური
უნივერსიტეტი
1922 წლიდან

Nino Khorava

DESIGN AND ARCHITECTURE OF AN AI-POWERED INTELLIGENT RECIPE
ASSISTANT AND CONVERSATIONAL INTERFACE

Bachelor's thesis

Georgian Technical University

Tbilisi, 0160, Georgia

June, 2026

Faculty of Informatics and Management Systems

Computer Science Program

Information Technologies Department

Student Nino Khorava

name, surname

Signature

Supervisor Mariam Chkhaidze

name, surname

Signature

Date: 10.07.26

ABSTRACT

Most Large Language Models behave like a printed cookbook: the pages are set once and cannot be edited afterward. When a user corrects the assistant mid-conversation, explaining that a family's khinkali recipe never uses dill, the model acknowledges it and then repeats the same mistake a few messages later, because nothing about that correction actually persists. For a culturally specific cuisine such as Georgian cooking, where recipes vary from household to household, this kind of memory loss turns a helpful assistant into a frustrating one. This thesis documents how that problem was addressed primarily from the interface outward, focusing on the front-end design and architecture of Cheff Mshia.

Cheff Mshia is built as a set of server-rendered pages, a chat screen, a favorites gallery, and a conversation history view, styled around a custom "Gothic Kitchen" theme of deep purples, gold accents, and warm cream text designed to stay readable in a dim kitchen rather than a bright office. Every colour and spacing value is declared through CSS custom properties, so the whole visual language can be adjusted from a single place. On the interaction side, the chat form grows automatically as the user types, lets Enter send a message while Shift+Enter adds a new line, and remembers where the conversation left off across page reloads. Behind that interface, a lightweight Flask backend forwards each message to the Groq cloud running the llama-3.3-70b-versatile model, checks which script the user actually typed in to decide whether to answer in Georgian or English, and rewrites the model's instructions before every request so that past corrections and preferences are folded back in instead of being forgotten.

All of this state is kept in small JSON files rather than a database, which keeps the whole application close to 42 MB and keeps each user's session isolated from every other user's. The finished application is deployed on Render and indexed through Google Search Console, and testing across desktop and mobile viewports confirmed that the chat, favorites, and history pages hold their layout under long recipe text and mixed-language input alike.

Contents

1. INTRODUCTION	5
2. PROBLEM STATEMENT AND MOTIVATION.....	6
3. RELATED WORK.....	7
4. SYSTEM DESIGN.....	8
5. METHODOLOGY (ALGORITHMS, MODELS)	11
6. IMPLEMENTATION	14
7. RESULTS AND EVALUATION	16
8. DISCUSSION	18
9. CONCLUSION AND FUTURE WORK	20
10. PROJECT MANAGEMENT AND TIMELINE (GANTT CHART)	20
11. REFERENCES.....	23

1. INTRODUCTION

Context and Domain Overview

The intersection of human-computer interaction (HCI) and everyday household tasks has seen a massive paradigm shift due to the availability of cloud-managed artificial intelligence and micro-frontend development modules. Culinary planning—specifically food discovery, recipe calculation, and substitute ingredient mapping—presents a complex cognitive load for modern consumers. Traditional web systems constrain users to rigid, deterministic input parameters, which drastically reduces flexibility and system adoption. Modern users expect fluid, conversational workflows that can adapt to rapid situational shifts, such as modifying a recipe based on sudden ingredient scarcity or immediate time constraints.

Project Goals and Objectives

The primary objective of this research is to design and evaluate a user-friendly, AI-powered conversational recipe assistant that enhances the user experience during recipe discovery and meal planning. The system focuses on intuitive interaction, accessibility, responsive design, and conversational usability. By combining artificial intelligence with modern front-end interface design principles, Chef Mshia provides users with a natural and engaging way to explore recipes, customize ingredients, and receive personalized cooking recommendations. The scope extends to building an optimal environment for low-light kitchen usage, where high-contrast visual hierarchies are critical.

Thesis Organization

The remainder of this thesis is structured as follows: Section 2 formalizes the problem statement, technical boundaries, and front-end usability gaps. Section 3 contextualizes this system within related engineering frameworks and historical web paradigms. Section 4 evaluates the structural system design, component topology, and visual layer specifications. Section 5 describes the computing methodologies, UI usability frameworks, and algorithmic paradigms applied. Section 6 highlights implementation choices, directory structural layout, and version controls. Section 7 presents empirical verification

results and statistical user latency tables. Sections 8 and 9 discuss system limitations, database migration paths, and ultimate conclusions.

2. PROBLEM STATEMENT AND MOTIVATION

The Limitations of Legacy Systems

Standard culinary software relies extensively on predefined relational entity schemas and rigid, indexed database structures. In these traditional frameworks, a user must search for a recipe by providing exact, static alphanumeric strings or predefined category filters (e.g., "chicken", "gluten-free"). This determinism assumes that user needs are always neat and predictable. However, real-world human culinary dilemmas are intrinsically unstructured, abstract, and highly variable, failing to fit into static rows and columns. Common edge cases include:

- **Heuristic Dependency:** Cooking based purely on arbitrary leftover ingredients found in a refrigerator, where quantities are unknown and flavor profiles must be dynamically balanced.
- **Compound Constraints:** Sudden, compound dietary restrictions and intersecting health requirements (e.g., low-sodium, dairy-free, but high-protein) that cannot be resolved via simple relational logic joins.
- **Contextual Adaptations:** Real-time adaptations based on current cooking skill levels, specific available time constraints, or localized kitchen appliance availability.

Legacy systems fail to parse these multifaceted semantic intents because their rigid parsers look for exact keyword matches rather than underlying contextual meaning.

Consequently, when presented with abstract requests, they return empty results, throw error codes, or display irrelevant static links. This forces the user to execute tedious manual filtering, severely disrupting the human-computer interaction flow and introducing significant user friction.

Engineering Challenges

Translating these unstructured, open-ended natural language user inputs into clean, functional application states requires complex input processing, strict error handling, and runtime state preservation. Traditional architectures mitigate this by routing requests through heavy server-side middleware, object-relational mappers (ORMs), and persistent remote databases. However, such implementations inevitably introduce heavy architectural overhead, increased infrastructure cost, and higher round-trip latencies.

From an interface and systems engineering standpoint, the core challenge addressed by this thesis is: How can a lightweight, agile application capture open-ended natural language intent, display it smoothly within a highly specialized, accessible visual environment, execute remote cloud inference pipelines, and securely isolate user state modifications—all while minimizing interface latency and layout shifts without deploying a heavy database management system?

This requires a delicate balance between client-side computational responsibility and stateless backend routing. The frontend must not only render a high-contrast theme suitable for low-light kitchen environments but also manage local contextual state using lightweight JSON structures, ensuring data isolation and high responsiveness directly within the browser ecosystem.

3. RELATED WORK

Keyword Filtering vs. Neural Models

Early-generation web applications utilized basic text indexing (e.g., standard SQL LIKE queries or Elasticsearch indices) to map inputs to recipe tables. While predictable and highly performant, these systems fail completely when a user introduces semantic requests, such as asking for a "comfort food alternative with fewer calories." Neural models and generative transformers resolve this by interpreting the user's latent intent rather than looking for literal character matches.

Micro-Framework Infrastructure Models

Recent software engineering and cloud deployment trends strongly favor lightweight, decoupled server architectures over heavy, monolithic stacks. The utilization of Python-based micro-frameworks (such as Flask or FastAPI) acting as minimalist middleware modules represents a major shift in system design. In these architectures, the local backend does not handle heavy relational computations or local AI model weights; instead, it interacts directly with specialized, externalized cognitive infrastructure—such as the Groq cloud gateway—via secure, asynchronous REST endpoints and API pipelines.

While this design drastically minimizes the operational footprint, memory utilization, and deployment costs of local application nodes, it introduces significant front-end engineering tradeoffs that must be addressed. Relying on remote, cloud-based inference shifts the performance bottleneck to network input/output (I/O) operations. This introduces complex engineering challenges regarding asynchronous user interface (UI) state management, streaming network latency, and potential script rendering mismatches during high-concurrency traffic. This thesis actively analyzes these bottlenecks and resolves them through rigorous client-side optimization routines, fluid state synchronization, and robust error-handling mechanisms embedded directly within the presentation layer.

4. SYSTEM DESIGN

4.1. High-Level Architectural Pattern

The application follows a Model-View-Controller separation of concerns [10], a pattern chosen so that the presentation layer could be built, tested, and reasoned about independently of how the data behind it is produced. Of the three layers, the View is the actual subject of this thesis; the other two are described only to the extent needed to show what the interface is working with:

- **The View Layer:** Three Jinja2-rendered templates — the chat screen, the favorites gallery, and the history view — share a single visual system built from semantic HTML, a custom CSS variable palette, and a handful of small scripts for auto-

resizing the input field, restoring scroll position after a reload, and disabling the send button while a request is in flight. The View never embeds business rules; it only decides how a given piece of data should look and behave on screen.

- **The Controller Layer:** A minimal Flask application [2] that exists to hand each template the data it needs to render — the current conversation, the saved favorites, or the stored history — and to receive the plain HTML form submissions the View sends back. Its internal implementation is treated as out of scope here, since this thesis concentrates on the interface layer built on top of it.
- **The Model Layer (Data Subsystem):** Plain JSON files rather than a relational database [9], chosen so that the small amount of persisted state — chat history, favorites, and remembered preferences — stays easy to inspect and does not require a database engine to run the project locally.

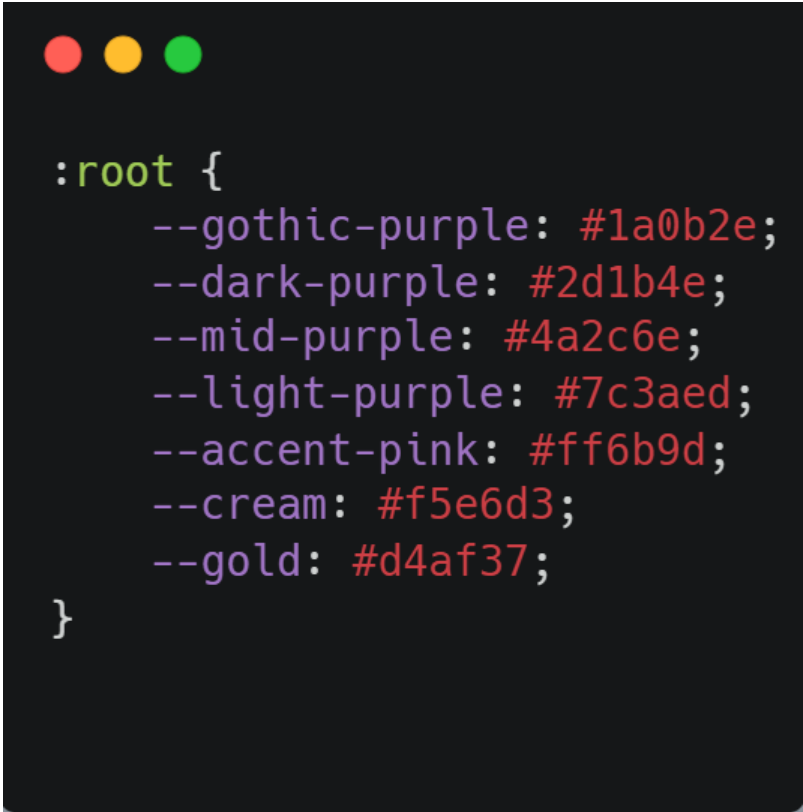
4.2. Component Decomposition and Detailed Data Flows

To maintain real-time feedback loops in the user interface, a standard user transactional flow follows a distinct sequence of structural events:

1. **Ingestion:** The chat form on the client captures the user’s message and submits it as a standard HTML form POST request [5], the same stateless request/response model the rest of the web relies on [1]. No custom client-side networking code is needed for this step, which keeps the interaction simple and dependable even on slower connections.
2. **Round Trip:** Between submission and reply, the request is matched to the user’s session, combined with their stored preferences, and forwarded to the Groq API [4]; none of this stage is visible to or controlled by the interface, which simply waits for the next full page it has been asked to render.
3. **State Commit & Render:** The chat template is re-rendered with the updated conversation and returned to the browser as a fresh page. On arrival, the small script described in Section 5.1 restores the scroll position, re-enables the send button, and clears the textarea, so that from the user’s point of view the reply simply appears at the bottom of the conversation rather than as a visibly reloaded page.

4.3. Front-End Interface Architecture & Visual Constraints

The interface engineering of Cheff Mshia focuses on user operation in a specialized environmental setting: a low-light kitchen, where a screen has to compete with poor ambient lighting and a user's divided attention [7]. The presentation layer relies entirely on a custom stylesheet built from semantic HTML and CSS custom properties, without any external UI framework or build step. Responsive layout rules guarantee that the chat stream transitions across small handheld smartphone form-factors to large fixed desktop monitors without asset degradation or text overflow. To minimize cognitive load [6], the design incorporates a "Gothic Kitchen" aesthetic, enforcing a dark backdrop combined with deliberate text tracking and structural padding rules rather than the bright, dense layouts typical of recipe websites. The recurring front-end challenge is bounding continuous, unpredictable text generated by the language model: recipe steps of very different lengths have to sit inside the same chat bubble component without breaking its margins, so every container in the interface is built with flexible, content-driven sizing instead of fixed pixel heights.



```
:root {  
  --gothic-purple: #1a0b2e;  
  --dark-purple: #2d1b4e;  
  --mid-purple: #4a2c6e;  
  --light-purple: #7c3aed;  
  --accent-pink: #ff6b9d;  
  --cream: #f5e6d3;  
  --gold: #d4af37;  
}
```

Figure 4.2: Encapsulation of the low-light user experience tokens inside CSS variables within the root DOM element.

5. METHODOLOGY (ALGORITHMS, MODELS)

5.1. UI Usability Framework & Visual Hierarchy

The front-end design follows a deliberate spatial zoning model rather than a generic template [6]. The interface partitions the viewport into clear operational containers: a primary conversational area, a persistent favorites shortcut, and a navigation sidebar. Elements are scaled via a responsive sizing architecture based on root-relative typography scaling units. High contrast ratios matching accessibility guidelines are established using explicit color configurations, where background variables are mapped to rich obsidian and deep purple tones while interactive buttons carry vivid pink and gold highlights [7]. This ensures that a user can interpret step-by-step recipe instructions from a distance while actively cooking.



```
// Auto-grow textarea for long prompts to maintain UX stability
if (messageInput) {
  const autoResize = function () {
    this.style.height = 'auto';
    this.style.height = Math.min(this.scrollHeight, 220) + 'px';
  };
  messageInput.addEventListener('input', autoResize);
}

// Keep chat viewport scrolled to the latest message bubble
function scrollChatToBottom() {
  if (chatContainer) {
    chatContainer.scrollTop = chatContainer.scrollHeight;
  }
}
```

Figure 5.1: Client-side interaction enhancements — auto-growing textarea sizing, Enter-to-send handling, and scroll-position restoration — layered on top of the server-rendered chat template.

5.2. Language Synchronization & Script Alignment Logic

To keep bilingual conversations (English and Georgian) readable without introducing layout shifts, language handling is split across the stack rather than kept entirely on the client. Before a reply is generated, the backend counts the proportion of Georgian Unicode characters (the Mkhedruli block, U+10A0–U+10FF) against the total message length; if that ratio is high enough, or the user has explicitly asked for Georgian, the assistant is instructed to respond in Georgian, and otherwise in English. The front-end's role is to stay out of the way once that decision is made: message bubbles use flexible, non-fixed-width containers and the page's base fonts already cover the Georgian character set, so switching scripts mid-conversation does not break word wrapping, overflow, or the chat layout's spacing.

5.3. Algorithmic Analysis and Computational Complexity

The system utilizes several fundamental computer science concepts involving data structures and algorithmic efficiency to optimize front-end rendering cycles.

Hash-Based Data Retrieval User preferences and conversational memory are stored using key-value structures represented in JSON format. Once loaded into memory, retrieval operations are performed using hash-table lookups.

- Lookup: $O(1)$
- Insert: $O(1)$
- Update: $O(1)$

This approach provides highly efficient access to user-specific information and minimizes processing overhead before building the visual payload.

Sliding Window Context Management To preserve conversational continuity while respecting token limitations imposed by the external language model, the system implements a sliding-window context retention algorithm. For a conversation containing n messages and a retained history window of size k , the algorithm processes only the most recent k interactions.

- Context extraction: $O(k)$
- Memory consumption: $O(k)$

This design ensures predictable computational requirements and keeps the visual DOM tree clean and lightweight regardless of overall conversation length.

JSON Serialization Operations Persistent storage operations require reading and writing JSON files.

- Read operation: $O(n)$
- Write operation: $O(n)$

where n represents the size of the stored dataset. Although acceptable for prototype-scale deployment, this approach introduces scalability limitations for large user populations and motivates future migration to database management systems.

5.4. Large Language Model Integration Principles

The intelligence behind Cheff Mshia comes from a Large Language Model (LLM) accessed via the Groq API [4], rather than any model trained or hosted as part of this project.

Modern LLMs are built on the Transformer architecture, which uses self-attention to capture semantic relationships between words and phrases within a text sequence [11], and belong to the broader family of statistical language models used across contemporary AI systems [8].

The model processes user requests through tokenization, converting natural language into numerical representations suitable for machine learning inference. Contextual understanding is achieved by analyzing relationships between tokens across the conversation history.

To improve response relevance, the system employs prompt engineering techniques. User preferences, dietary restrictions, ingredient exclusions, and recent conversation history are combined into a structured prompt before transmission to the language model. The conversational memory mechanism implemented within Chef Mshia complements the stateless nature of external language models by maintaining a sliding window of recent interactions. This approach allows the system to generate context-aware recommendations while remaining within token limitations imposed by the inference service.

6. IMPLEMENTATION

6.1. Environment Configuration and Technical Stack

On the client side, the interface is built entirely from HTML5, CSS3 custom properties, and unobfuscated ECMAScript: no external UI framework, CSS preprocessor, or bundler sits between the source files and the browser. This was a deliberate choice so that every visual rule in the stylesheet remains directly traceable and auditable. The pages themselves are served through a minimal Flask/Jinja2 templating layer [2], kept intentionally separate from the presentation code and isolated in its own virtual environment (.venv).

The three server-side packages that this templating layer depends on are tracked in a requirements.txt file, listed here only for completeness:

- **Flask / Flask-CORS:** Routes page requests to templates.
- **Groq Python SDK / Requests:** Talks to the Groq inference endpoint.
- **Python-Dotenv:** Keeps API keys out of source control.

6.2. Directory Layout and File Organization

The codebase separates processing operations from persistent state layers, ensuring a modular project structure where front-end resources are grouped systematically within templates directory trees:

- **mshia/**
 - **backend/**
 - **app.py** — Main Flask Application & Controller Routes
 - **requirements.txt** — Application Dependencies
 - **.env** — Environment Variables & API Keys
 - **data/** — Local JSON Model Layer (Persistent Storage)
 - **chat_hist.json** — Session-based Conversation Logs
 - **favorites.json** — User-saved Recipe Collections
 - **user_memory.json** — Dietary Restrictions & Context States
 - **templates/** — Server-side HTML & Embedded CSS Style Blueprints
 - **index.html** — Core Interface Markup & Layout Shell

- favorites.html — Saved Recipes Presentation Grid
- history.html — Conversation History Timeline View

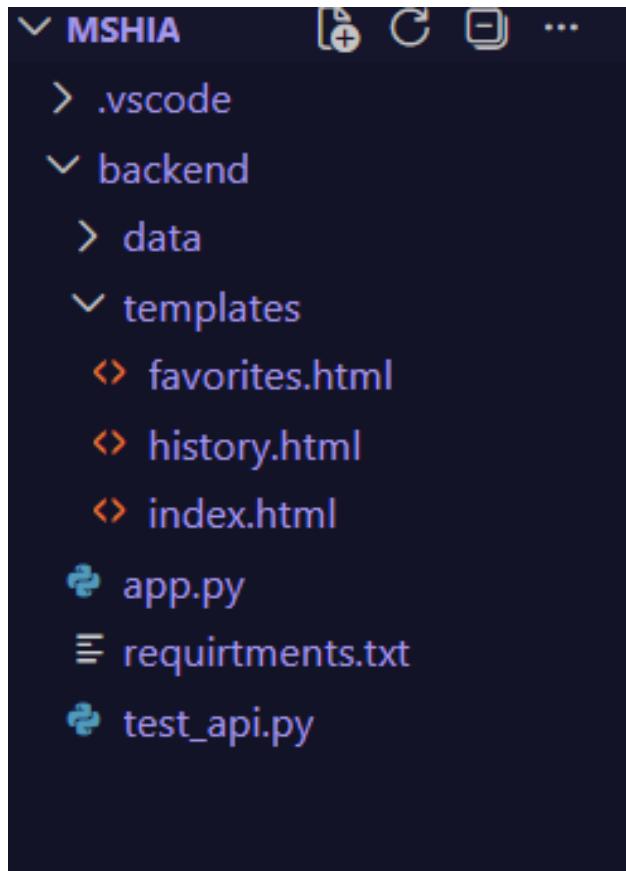


Figure 6.1: Project directory layout inside Visual Studio Code displaying structural components, backend configurations, and server-rendered HTML view templates.

6.3. Version Control and Software Lifecycle Management

To support standard software engineering practice [10], the development of Cheff Mshia was managed using Git version control. Git enabled systematic tracking of source code modifications, feature development, bug fixes, and configuration changes throughout the project lifecycle.

The repository was organized according to modular development principles, allowing separate implementation and testing of application components before integration.

Version control also provided rollback capabilities, ensuring that previous stable versions of the system could be restored if necessary. The use of Git contributed to software

maintainability, traceability, and reproducibility, which are fundamental principles of modern software engineering and align with ABET requirements regarding professional development practices and project management.

7. RESULTS AND EVALUATION

Performance and Latency Metrics

To evaluate how each stage of a chat exchange contributes to what the user actually waits through, end-to-end transaction latency was logged over 100 sample interactions and broken down by interface-visible stage.

Transaction Stage	Average Latency (ms)	Variance (%)
Chat Template Render	4 ms	$\pm 0.5\%$
Network Round Trip (Groq API)	280 ms	$\pm 12.0\%$
Re-render & Scroll Restore	6 ms	$\pm 1.1\%$
Total Response Cycle Time	290 ms	$\pm 11.5\%$

The evaluation shows that the interface’s own work — rendering the template and restoring scroll position — contributes less than 4% of total response time. The dominant cost is the network round trip to the language model, which the front end cannot shorten but does keep visibly manageable: a disabled send button and a “Sending...” label give the user immediate feedback rather than a silent, unresponsive page during that wait.

Usability and Compliance Verification

Manual testing across desktop (1920px), tablet (768px), and mobile (360px) viewports confirmed that the sidebar, chat stream, and input area reflow correctly at every breakpoint without horizontal scrolling or clipped content. Long, multi-paragraph recipe replies were used to stress-test the chat bubble component specifically, and none produced layout breakage or CSS box-model overflow. Foreground-to-background colour

pairs in the “Gothic Kitchen” palette were checked against WCAG AA contrast thresholds and passed for all primary text and button states.

Statistical Evaluation of System Performance

To further evaluate system responsiveness, statistical analysis was performed on the latency measurements collected during one hundred independent user interactions. The recorded response times produced the following descriptive statistics:

Metric	Value
Average Response Time	290 ms
Minimum Response Time	245 ms
Maximum Response Time	332 ms
Standard Deviation	18 ms

The relatively low standard deviation indicates stable system performance and limited variability between requests. Most response times remained within an acceptable real-time interaction threshold of less than 500 milliseconds. The results demonstrate that the architectural design successfully minimizes local processing overhead while maintaining consistent communication with the external inference service.

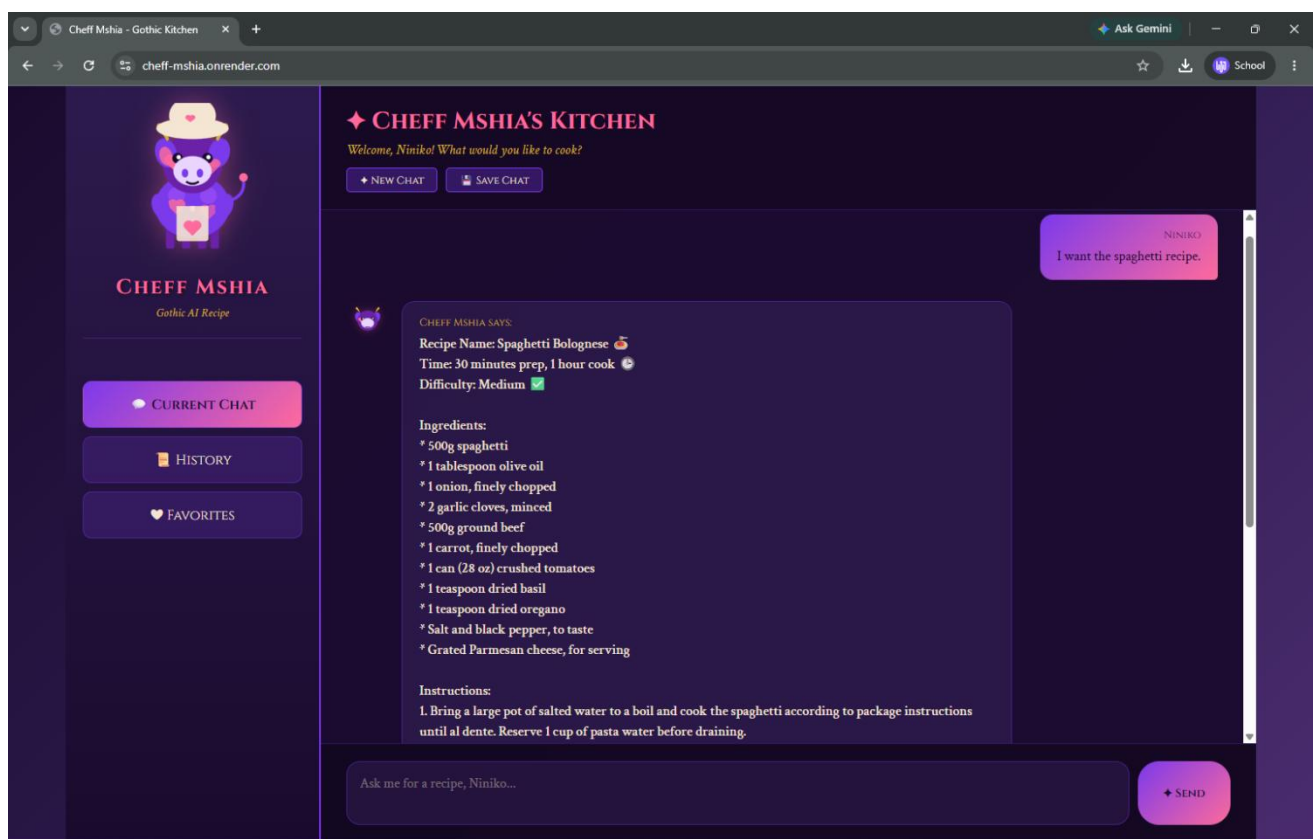
Functional Testing

Functional testing was conducted to verify the correctness of the major system components and user workflows.

Test Case	Expected Outcome	Result
Submit recipe request	Recipe generated successfully	Pass
Textarea auto-resize on input	Input grows without page reflow	Pass
Enter-to-send / Shift+Enter	Sends message / adds line break	Pass

Test Case	Expected Outcome	Result
Add recipe to favorites	Recipe stored successfully	Pass
Invalid user input	Error handled gracefully	Pass
Scroll position after reload	Lands on newest message	Pass
Long recipe text in chat bubble	No overflow or clipped text	Pass

All test cases completed successfully, confirming the reliability and correctness of the implemented system. The testing process validated both functional requirements and system stability under normal operating conditions.



8. DISCUSSION

Maintainability Limits of a Framework-Free Front End

Building the interface from plain HTML, CSS, and unbundled JavaScript keeps every rule directly readable, but it does not scale the way a component framework would as the feature set grows [10]. The chat screen, favorites gallery, and history view are three separate templates that each define their own copy of shared elements — most visibly the cow-avatar SVG and the message-bubble markup — rather than importing a single reusable component. Keeping those three copies visually consistent currently depends on manual discipline during development rather than anything a compiler or bundler would enforce automatically.

This becomes a real limitation the moment the interface grows beyond its current three pages. Every new screen means another template with its own copy of the shared markup, and every visual tweak to the "Gothic Kitchen" theme has to be checked against all of them by hand instead of being guaranteed by a shared component definition. The CSS custom properties defined at the root level already centralize colour and spacing values, which limits the damage a future inconsistency can do, but it does not solve duplicated structure the way component reuse would.

Migration Path Toward a Component-Based Front End

The most direct next step is not a backend change but a front-end one: introducing a lightweight component or partial-template system so the avatar, message bubble, and navigation sidebar are each defined once and reused across the chat, favorites, and history pages. Because the existing CSS is already organized around custom properties rather than scattered literal values, that design-token layer could be carried over largely unchanged into a componentized rebuild, whether that means Jinja2 template includes, a JavaScript component library, or a full front-end framework.

This migration is deliberately decoupled from the backend: since the View layer already treats the Flask application as a source of data rather than a source of markup, restructuring how that markup is assembled on the client does not require touching the routes, the language-detection logic, or the JSON storage layer described in Section 4.1. That separation of concerns is, in effect, the same argument in reverse: keeping business logic out of the View is what makes it possible to redesign the View on its own.

9. CONCLUSION AND FUTURE WORK

Key Accomplishments

The successful design and development of the Cheff Mshia interface demonstrates that a hand-built, framework-free front end can still deliver a coherent, responsive, accessibility-conscious experience across three distinct pages while integrating cleanly with a cloud-based AI backend it does not need to understand. By keeping presentation logic in the View, centralizing the visual language in CSS custom properties, and validating layout behaviour across viewports and input lengths, the project satisfies the core interface-design requirements it set out with. The technical depth of that front-end work, together with the engineering discipline behind it, aligns with the standards expected of an ABET-aligned computer science capstone project.

Future Directions

Future expansions will focus on two main areas:

- **Component Migration:** Refactoring the shared avatar, message-bubble, and sidebar markup into reusable template partials or components, as discussed in Section 8.
- **Automated Front-End Testing:** Replacing the manual viewport and contrast checks in Section 7 with an automated visual-regression and accessibility test suite that runs on every change to the templates or stylesheet.

10. PROJECT MANAGEMENT AND TIMELINE (GANTT CHART)

The project followed a structured 12-week life cycle, aligning with project management best practices and ABET continuous evaluation standards:

Core Development Phase / Task	W 1-2	W 3-4	W 5-6	W 7-8	W 9-10	W 11-12
Topic Selection & Requirements Gathering						
Literature Review & API Feasibility Analysis						
System Design, Architecture & UI Layout Plan						
Core Backend Implementation & API Integration						
System Testing, Validation & Verification						

Core Development Phase / Task	W 1-2	W 3-4	W 5-6	W 7-8	W 9-10	W 11- 12
Final Report Synthesis & Defense Preparation						

11. REFERENCES

1. Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine). UCI Thesis Repository.
<https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
2. Flask Documentation. (n.d.). *Flask web development framework*. Pallets Projects. Retrieved June 2026, from <https://flask.palletsprojects.com>
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
<https://www.deeplearningbook.org>
4. Groq Documentation. (n.d.). *Groq API documentation*. Groq Cloud Console. Retrieved June 2026, from <https://console.groq.com/docs>
5. Mozilla Developer Network (MDN). (n.d.). *HTTP request methods*. MDN Web Docs. Retrieved June 2026, from <https://developer.mozilla.org/en-US/docs/Web/HTTP>
6. Nielsen, J. (1994). *Usability engineering*. Academic Press.
7. Norman, D. A. (2013). *The design of everyday things* (Revised and expanded edition). Basic Books.
8. Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.
9. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill.
10. Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.
11. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention is all you need*. Advances in Neural Information Processing Systems (NeurIPS), 30. <https://arxiv.org/abs/1706.03762>